

Laboratorio individuale - Industry

GHERARDI GIACOMO S4235299

Funzioni richieste implementate

- createEmptyIndustry **OK**
- insertBasicItem **OK**
- insertItem **OK**
- isPresentItem **OK**
- removeItem **OK**
- addBasicItem **OK**
- listNeed **OK**
- listNeededBy **OK**
- listNeededByChain **OK**
- howManyItem **OK**

Funzioni ausiliarie implementate

- **findItemIndex** : serve per trovare l'indice di un item nell'array dato il suo nome.
- **hasPath**: funzione che verifica tramite DFS se esiste un percorso nel grafo delle dipendenze tra due item. Viene chiamata prima di inserire un nuovo item composto per assicurarsi che la nuova dipendenza non crei un ciclo.
- **getDeps**: funzione ricorsiva che esegue la navigazione in profondità del grafo per esplorare l'intera catena di dipendenze.
- **findToDelete**: funzione che raccoglie tutti gli item che devono essere rimossi quando si elimina un item (inclusi tutti i dipendenti)
- **getBasicItem**: calcola ricorsivamente la quantità di Basic item necessaria a produrre un'unità di item composto; uso una map per sommare le quantità richieste.
- **sortList**: implementa un quicksort per ordinare le liste.

Ho deciso di implementare le seguenti librerie:

- **map**: utile per calcolare i requisiti in howManyItem
- **set**: mi permette di gestire collezioni ordinate senza duplicati durante le operazioni di visita del grafo. Ho utilizzato i set per:
 - Tenere traccia dei nodi visitati nelle funzioni di ricerca
 - Raccogliere gli item da rimuovere senza duplicazioni
 - Costruire le liste di risultato
- **climits**: mi è servito per usare la costante UINT_MAX all'interno della funzione howManyItem per inizializzare la variabile minProd, che tiene traccia del numero min di item producibili.

Per compilare, ho utilizzato il comando **g++ *.cpp** nella cartella principale.

Eseguendo il codice, ottengo che tutti i test ha come risultato PASSED:

```
-----  
-----> NUMBER OF TESTS PASSED: 218/218  
-----
```